



PyTorchSim 의 TensorFlow 확장

학 번: 20220312
이 름: 박준혁
연구 지도교수: 김광선
학 과: 컴퓨터공학과

연구 목적 (Problem statement)

최근 인공지능 모델의 복잡도가 기하급수적으로 증가함에 따라, 이를 효율적으로 처리하기 위한 신경망 처리 장치(NPU, Neural Processing Unit)의 아키텍처 연구 및 하드웨어-소프트웨어 공동 설계의 중요성이 대두되고 있다. 이러한 연구를 뒷받침하기 위해 사이클 수준의 정확도를 제공하는 시뮬레이터의 존재는 필수적이며, 현재 POSTECH PSAL Lab 에서 구축한 NPU 시뮬레이터인 PyTorchSim[1]은 PyTorch[2] 생태계에 강하게 결합되어 있어, 범용적인 활용에 제약이 따르고 있다.

인공지능 연구 및 산업계는 PyTorch 뿐만 아니라 TensorFlow [3], JAX[4] 등 다양한 딥러닝 프레임워크를 혼용하여 사용하고 있다. 새로운 프레임워크에서 작성된 모델을 검증하기 위해 타겟 아키텍처에 맞춘 백엔드 코드 생성기나 런타임을 매번 완전히 새롭게 개발하는 것은 막대한 시간과 개발 비용을 소모하게 만든다. 따라서, 본 연구는 개별 프레임워크에 종속되지 않는 범용적인 중간 표현(IR, Intermediate Representation)인 HLO(High-Level Optimizer)를 매개로 활용하여, TensorFlow 환경을 기존 PyTorchSim 구조에 매끄럽게 통합하고자 한다. 이를 통해 단일 시뮬레이터 인프라로 다중 프레임워크를 지원하는 고도화된 연구 환경을 구축하고자 한다.

연구 배경 (Motivation and background)

현대 딥러닝 프레임워크들은 각자의 고유한 컴파일 파이프라인과 중간 표현을 통해 연산을 최적화하고 하드웨어에 배포하고 있다. PyTorch 의 경우, PyTorch 2.0 부터 도입된 Torch FX Graph 의 형태로 파이썬 수준의 모델 연산 그래프를 포착(Capture)하고, 이를 TorchInductor Backend 에서 연산을 Fusion 하고 C++, Triton 코드로 변환(Lowering)하는

구조를 가진다. PyTorchSim 에서는 Custom TorchInductor Backend 를 구현, NPU 시뮬레이터 특화 MLIR 로 Lowering 하는 부분이 구현되어 있다.

반면 TensorFlow 의 경우 GraphDef 의 형태로 파이썬 모델의 그래프를 캡처하고, XLA(Accelerated Linear Algebra) 컴파일러를 사용해 머신 코드로 변환한다. 이 과정의 결과물은 Machine-Independent 한 HLO(High-Level Optimizer) 코드이며 이후 Backend 를 통해 머신 코드로 변환한다. 이에 본 연구에서는 HLO 코드로부터 기존의 NPU 시뮬레이터 특화 MLIR 로의 커스텀 패스를 구현하고자 한다. 이때 Tensorflow 생태계에 새로운 하드웨어 가속기를 연동하기 위한 방법 중 PJRT(Pretty much Just another RunTime)[5]를 채택하여 XLA Compiler 과 PytorchSim 을 연결한다.

연구 방법 (Design and methodology)

본 연구는 HLO 와 PJRT 를 활용하여 TensorFlow 파이프라인과 PyTorchSim 을 통합하기 위해 다음과 같은 네 가지 핵심 단계로 나뉘어 연구를 수행한다.

1. TensorFlow 구조 연구 및 분석

TensorFlow 모델이 Graph Mode 로 실행될 때 어떤 과정으로 Machine Code 까지 생성되는지 면밀히 분석하고, PyTorchSim 구조와 비교분석하여 적절한 통합 방법을 찾는다.

2. PJRT 구현 및 연결

TensorFlow 컴파일러 파이프라인에 개입하기 위해 PJRT API 를 개발하고 이를 TensorFlow 런타임에 커스텀 백엔드로 등록하여, 모델 실행 시 컴파일러가 생성한 HLO 그래프가 기본 하드웨어가 아닌 본 연구의 플러그인으로 라우팅되도록 한다.

3. Codegen 구현

PJRT 플러그인을 통해 전달받고 파싱된 HLO 연산들로부터 Custom MLIR Code 를 만드는 Codegen 메커니즘을 구현한다. 이후 시뮬레이터 부분과 연결하여 기존의 PyTorchSim 과 동일한 동작을 하도록 한다.

4. 성능 테스트 및 기존 시뮬레이터와 비교

구현된 통합 시스템의 검증을 위해 CNN, Transformer, LLM 모델 등 대표적인 딥러닝 벤치마크 모델을 구동하고, 실행 정확도/사이클 수/메모리 사용량을 비교 분석하여 전체적인 시뮬레이션의 신뢰성과 효율성을 검증한다.

연구 진행 상황 (Progress report)

I. 아키텍처 분석 및 채택된 구현 방식

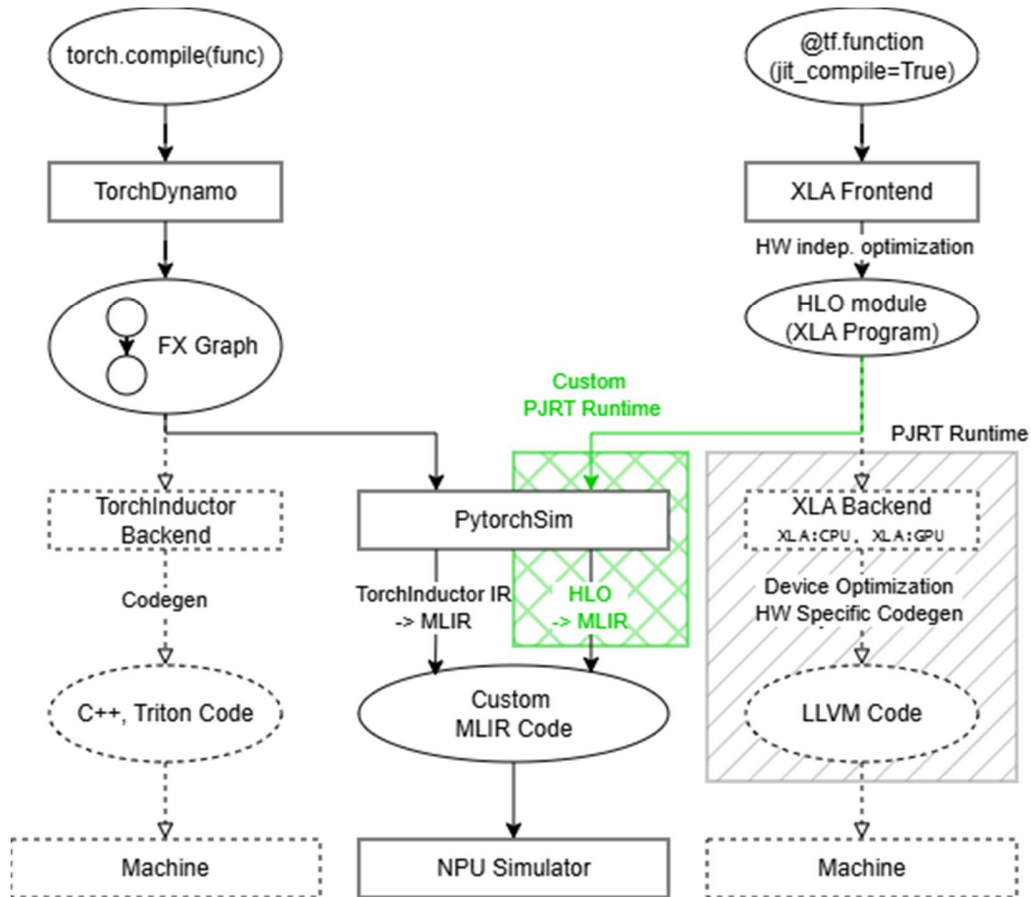


그림 1 PyTorch/TensorFlow 실행 구조와 본 연구의 구현 범위 (초록색)

TensorFlow 의 Graph mode execution 은 다음과 같이 진행된다.

1. XLA Compiler frontend 에서 HLO Module 을 생성한다.
 - A. 이때 HW Independent Optimization 을 진행하게 된다.
2. 생성된 HLO Module 은 PJRT Runtime API 를 통해, Backend 로 전달된다.
 - i. 기존 방식: XLA:CPU, XLA:GPU 등의 XLA Backend 로 전달되어 HW Dependent Optimization 을 거쳐 codegen 을 통해 CPU/GPU 로 전달된다.
 - ii. 본 연구에서의 방식: PytorchSim 으로 전달되어, 기존에 구현된 대로 NPU Simulator 의 MLIR Code 가 생성된다.

II. 구현된 부분

- PluggableDevice 를 사용한 시뮬레이터 인식 테스트

```
2026-02-09 05:45:59.523160: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.  
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.  
2026-02-09 05:46:05.317105: E external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:51] failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN ERROR (303)  
[PhysicalDevice(name='/physical_device:CPU:0', device_type='CPU'), PhysicalDevice(name='/physical_device:MY_DEVICE:0', device_type='MY_DEVICE')]
```

그림 2 PluggableDevice 테스트. 쉬운 구분을 위해 MY_DEVICE 라는 이름을 사용하였다.

- 본 연구에서는 새로운 NPU Simulator Device 를 TensorFlow 가 인식할 수 있도록 해야 한다. 따라서 PluggableDevice[6]을 활용하여 C API 기반으로 커스텀 디바이스를 추가하는 데에 성공하였다.
- 본 연구에서 사용할 PJRT 는 PluggableDevice 와 비슷한 디자인을 채택하고 있기 때문에 [7], 비슷한 방식으로 PJRT Runtime 에서 추가할 수 있을 것으로 기대된다.

-

III. 구현해야 하는 부분

1. Codegen: 기존의 PytorchSim 은 Loop-Level TorchInductor IR (~50 ops)에서 Custom MLIR 을 생성하는 Codegen 이 구현되어 있다. XLA 생태계로 확장하기 위해서는 HLO 에서 Custom MLIR 을 생성하는 과정이 구현되어야 한다.
 - A. HLO 는 StableHLO 로 자유롭게 변환될 수 있으며 StableHLO 는 약 100 개의 operation 을 가지고 있다.
 - B. 따라서 HLO->StableHLO->Custom MLIR Codegen 을 작성할 것이며, 이때 기존의 TorchInductor IR 변환 코드를 최대한으로 재활용할 수 있을 것이다.
2. PJRT Runtime
 - A. 기존의 구현들을 참고하여[8] Custom PJRT Runtime 을 구현한다.
HLO Module 을 입력으로 받아 Custom MLIR Code 를 출력하는 구조가 된다.
 - B. 이때 기존의 PytorchSim 이 Python 으로 작성되어 있고, TorchInductor Backend Pass 에 강하게 결합되어 있기 때문에 진입점을 새로 만들거나 dummy python script 를 작성하여 호출하여야 한다.

연구 추진 일정 (Future plan)

기간	내용	비고
4/25~5/15	PJRT Plugin 구현 및 연결 Codegen 구현	5/4 중간발표 녹화 5/8 중간발표 동료평가
5/15~5/29	PytorchSim 과 연결, 디버깅 성능 테스트 및 기존 시뮬레이터와 비교	5/29 최종발표, 최종 보고서

참고 문헌

- [1] Yang, W. a. (2025). PyTorchSim: A Comprehensive, Fast, and Accurate NPU Simulation Framework. Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture, 1363-1380.
doi:10.1145/3725843.3756045
- [2] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarakar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhersch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24), Vol. 2. Association for Computing Machinery, New York, NY, USA, 929–947. <https://doi.org/10.1145/3620665.3640366>
- [3] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng C. (2016). TensorFlow: a system for Large-Scale machine learning. “12th USENIX symposium on operating systems design and implementation”, 265-283.
- [4] Frostig, R., Johnson, M. J., & Leary, C. (2019, March). Compiling machine learning programs via high-level tracing. In SysML conference 2018.
- [5] <https://openxla.org/xla/pjrt>
- [6] <https://blog.tensorflow.org/2021/06/pluggabledevice-device-plugins-for-TensorFlow.html>
- [7] <https://opensource.googleblog.com/2023/05/pjrt-simplifying-ml-hardware-and-framework-integration.html>
- [8] <https://openxla.org/xla/pjrt/examples>